

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»

**институт информационных технологий и технологического образования
кафедра информационных технологий и электронного обучения**

Основная профессиональная образовательная программа
Направление подготовки 09.03.01 Информатика и вычислительная техника
Направленность (профиль) «Технологии разработки программного обеспечения»
форма обучения – очная

Курсовая работа

по дисциплине «Технологии компьютерного моделирования»

Моделирование компьютерной игры-стратегии с использованием элементов
искусственного интеллекта

Обучающегося 2 курса
Е.Е.Шульга



Руководитель:
доктор педагогических наук, профессор
_____ Е.З Власова

« _____ » _____ 2024 г.

Санкт-Петербург
2024

ОГЛАВЛЕНИЕ

Введение	3
Глава 1 Теоретический блок	
1.1 Описание стратегии настольной игры.	5
1.2 Обоснование выбранных материалов и ресурсов	8
1.3 Проектирование структуры и логики приложения	13
Глава 2 Практический блок	
2.1 Моделирование хода диалога бота	14
2.2 Разработка обработчиков событий, функций событий	15
2.3 Финальный код продукта. Метрика возможных ошибок	18
Заключение	20
Список литературы	21

ВВЕДЕНИЕ

Цель курсовой работы:

Методами языка программирования Python разработать приложение, работающее на основе чат-бота Telegram, которое реализует стратегию настольной игры «Мафия» на этапе «Знакомство игроков» с использованием элементов искусственного интеллекта OpenAI

Материалы и инструменты:

- - Python 3.12
- - Telegram Bot API
- - OpenAi API
- - Библиотеки python, необходимые для развертывания приложения:
 1. Sqlite3 – База данных для хранения сводных данных о пользователе и сессии игры
 2. OpenAI – Библиотека методов для работы с запросами к сервису «ChatGPT»
 3. Aioogram3 – Библиотека-конструктор чат-бота для Telegram
 4. Re – библиотека для парсинга сообщений с помощью регулярных выражений
 5. Random – модуль для внедрения элементов случайности
 6. Asyncio – Библиотека асинхронных методов для непрерывной работы приложения

Задача курсовой работы:

При выборе темы, я основывался на актуальности внедрения искусственного интеллекта в различные сферы и области приложений, к моменту написания курсовой работы, повсеместно разработчики внедряют его элементы для улучшения качества сервиса, упрощения навигации по клиенту приложения. Иногда, искусственный интеллект выступает, как ядро, без которого было бы невозможно выстроить правильную логику работы программы.

Моя курсовая работа – экскурс для таких же молодых разработчиков, как и я, по внедрению элементов искусственного интеллекта в собственные сервисы. Мною были определены следующие задачи:

- - Разработать стратегию настольной игры «мафия», на примере

которой, показать, как можно использовать ресурсы искусственного интеллекта

- - Разработать собственную базу данных для хранения сводных данных пользователя, сессии игры, для реализации «памяти» искусственного интеллекта
- - Прописать взаимодействие пользователя с искусственным интеллектом, продемонстрировать «мыслительный» процесс искусственного интеллекта при его взаимодействии в диалоге сразу с несколькими участниками в обстановке настольной игры «мафия».
- - Спроектировать метрику ошибок, которые могут возникать при работе с искусственным интеллектом, заложить в проект перспективу на дальнейшую поддержку и совершенствование

Важно!:

Сервисы, которые используются для реализации элементов искусственного интеллекта могут быть на текущий момент платными или же недоступными для некоторых регионов, для реализации сервисов автор рекомендует обратиться к отечественному разработчику, к примеру воспользоваться ресурсами «YandexGPT API» с бесплатным тарифом на запросы.

Приложение telegram бота будет развернуто локально на персональном компьютере автора, для бесперебойной работы проекта, рекомендуется использовать сервисы-хостинги. Для примера, в ходе курсовой работы, будет разобран хостинг приложения на бесплатном сервисе «Replit».

ТЕРМИНОЛОГИЯ:

Термин	Значение						
Промт	Запрос к сервису ChatGPT через библиотеку OpenAI, для получения ответа от искусственного интеллекта, содержит в себе сообщение, на основе которого строится ответ						
Карточка игрока	подразумевает сводные сведения о случайно распределенных параметрах для участника сессии, она выглядит и хранится в базе данных следующим образом: <table><tr><td>Очередь игрока</td><td>Имя персонажа</td><td>Роль персонажа</td><td>Масть персонажа</td><td>Память участника</td><td>Ключ сессии участника</td></tr></table> Где: ● ключ сессии - ID пользователя telegram ● Память участника – последние три сообщения, отправленные каждым активным пользователем сессии ● Роль персонажа – инструменты, доступные определенному участнику в рамках одной сессии ● Масть персонажа – отношение к категории роли, может быть: красная, черная	Очередь игрока	Имя персонажа	Роль персонажа	Масть персонажа	Память участника	Ключ сессии участника
Очередь игрока	Имя персонажа	Роль персонажа	Масть персонажа	Память участника	Ключ сессии участника		
Состояние	Особый фильтр для обработчиков, определенный через машину состояний библиотеки Aiogram в модуле FSM, состояния хранятся в отдельном файле states.py, в рамках одного состояния пользователь может перемещаться между экранизированными обработчиками и, пока пользователь не выйдет из состояния, код приложения будет повторяться						
Ведущий	Менеджер сессии, основанный на выстроенной логической цепочке функций python, включающий в себя методы базы данных, а также частные функции обозначенные в файле utils.py						

(Таблица 1.)

1.1 Описание стратегии настольной игры

ХОД ИГРЫ:

1) Подготовка к игре

Выбирается ведущий, обычно это один из опытных игроков. В нашем случае ведущий игрок – это телеграмм бот с выстроенной на Python логикой, далее в описании автор будет указывать ведущего, подразумевая именно эту дефиницию. Затем нужно определиться какие персонажи и модификации будут в игре. Это напрямую зависит от количества участвующих игроков. Приложение на текущий момент не использует модификации и реализует базовое наполнение игры в мафию, количество участников задается в начале каждой сессии пользователем, минимум – 6. Роли определяются случайно в следующем порядке:

- 1 комиссар

- 1 доктор

- мафия определяется формулой: $mafia = \frac{[общее количество участников]}{3,5}$ (1)

- мирные жители, это все остальные участники, то есть: $civ = [общее количество участников] - особые роли$ (2)

Определившись с составом персонажей, они замешиваются в случайном порядке и раздаются так, чтобы никто не знал чужих ролей.

2) Утро

Все игроки «просыпаются», случайным порядком задается порядок хода игрока, и по своей очереди игрок знакомится со «столом», в нашем случае при очереди игрока, пользователь вводит свое сообщения для анализа искусственным интеллектом, при очереди искусственного интеллекта, на сервер отправляется запрос с содержанием:

- [Имя], [роль], [масть карты], [количество участников], [имена участников], [память искусственного интеллекта]

3) Ночь

Ведущий объявляет наступление ночи, весь город засыпает. Далее ведущий оглашает ход персонажей в порядке, указанном ниже. Названный персонаж просыпается и выполняет действия, соответствующие его роли, после чего опять засыпает.

Порядок ходов ночью:

1. Мафия, если игрок, является членом банды мафии, то ему будет доступна история взаимодействия членов мафии, по очереди члены банды, в том числе игрок, выбирают номер игрока, которого хотят убить этой ночью, обосновывают свой выбор, если мафия в первый проход не пришла к общему выбору, игра повторно запрашивает номер участника сессии, указывая, что игроки не пришли к общему выбору в первый проход, если после этого игроки также не смогли определиться, приложение на основе выборов, случайным образом определяет цель для «убийства». Если игрок не является членом банды мафии, ведущий передает запросы к искусственному интеллекту со следующим содержанием:

- [Имя], [роль (мафия)], [количество участников], [имена участников], [память искусственного интеллекта], [выбранные другими членами мафии игроки и их обоснование]

Далее сервис проводит тот же порядок действий, что и при игроке-мафиози, но без доступа игроку к истории взаимодействий.

2. Доктор, ведущий запрашивает у участника номер игрока, которого он хочет защитить этой ночью, если выбор доктора совпал с выбором мафии, на следующий день игрок не умрет, лечить одного и того же игрока на протяжении нескольких ночей – Запрещено.

4. Комиссар, ведущий запрашивает у участника номер игрока, которого он хочет проверить этой ночью, при этом разглашается масть карты участника, а не его роль, если масть красная: проверенный игрок входит в команду мирных жителей, он может быть: доктором, комиссаром, мирным жителем, [другие особые роли в зависимости от модификаций]. Если масть черная, проверенный игрок входит в команду мафии или же другой команды-антагониста по отношению к мирным жителям, он может быть: мафией, [другой особой роли в зависимости от модификаций]

Примечание: доп. персонажи ходят в очередь, указанную в их описании.

После того, как все персонажи выполнили свою роль, ведущий объявляет наступление следующего дня. По правилам этикета в игру мафия:

- Запрещается явное озвучивание своих ролей или масти, чтобы не потерялся интерес к игре, разрешается только притворство, создание себе образа с какой-либо ролью, чтобы выстроить более сложную стратегию, для мафии, к примеру, может быть полезным представиться комиссаром, если подозрения

падают на участника, он заявляет о своей особой роли, его действиях предыдущей ночью. Для мирных жителей также, иногда, может быть полезным вскрыться мафией, чтобы отследить реакцию игроков. Также вскрытие роли предусмотрено при его логической необходимости, например, для шерифа это может быть обнаружение мафиози при проверке.

- Запрещается какое либо поведение ведущего, что может намекнуть участникам на роль кого-либо за столом, в случае автора это обеспечивается тем, что сводные сообщения от ведущего не хранятся в памяти искусственного интеллекта, экранированием сообщений от игрока в ночное время.

- Приветствуется мета-гейминг, этот термин обозначает внутриигровое общение, построенное на легенде игрового персонажа, в случае автора, заложена перспектива на внедрение поведенческих черт для искусственного интеллекта за счет модернизации промта*.

Для поставленных задач, логики работы к следующему дню не требуется, в проект заложены перспективы для реализации дальнейшей логики работы, в следующих версиях приложения автор внедрит:

логику переключения между состояниями день/ночь, дополнительные модификации, развлекательные сервисы для игрока, с определением поведенческих черт искусственного интеллекта и его характеристик.

1.2 Обоснование выбранных материалов и ресурсов

В ходе изучения материалов курсовой работы могут возникнуть вопросы, почему автор для реализации использовал именно эти сервисы, ниже будет представлено наименование продукта/ресурса и обоснование его использования.

1.2.1: почему выбрана технология чат-бота?

Клиентская часть приложения была реализована через библиотеку Aiogram 3 для языка Python, выбор был сделан исходя из практичности использования ресурсов компьютера, а также задач, которые были поставлены для программной части продукта, на текущий момент, большая часть общения и взаимодействия человека происходит через социальные сети, мессенджеры и, если рассматривать коммерческий продукт, конверсия при использовании структуры чат-бота для интерфейса такого приложения будет намного выше, нежели реализация через построение полноценного отдельной программы. Если рассматривать любой продукт-сервис, где

используются элементы искусственного интеллекта, с учетом имеющихся на текущий момент ресурсов мессенджера telegram, автор рекомендует использовать одноименное api.

При выборе библиотеки, которая будет предоставлять методы работы с telegram ботом, было выявлено два модуля-кандидата:

- Telebot
- Aiogram

Основные требования к библиотекам для телеграм-ботов

Модульность и расширяемость

Модульность - это ключевой аспект в этом контексте. Хорошая библиотека должна предоставлять удобные механизмы для разделения функциональности на отдельные модули, что позволяет легко добавлять новые возможности и взаимодействовать с различными частями бота.

Поддержка асинхронности

Асинхронное программирование становится все более важным аспектом при создании телеграм-ботов, особенно при обработке большого количества запросов. Профессиональные разработчики ищут библиотеки, которые предоставляют поддержку асинхронности, такие как asyncio. Это позволяет эффективно управлять множеством одновременных запросов без блокирования основного потока выполнения.

Удобство в использовании

Удобство в использовании включает в себя хорошо документированные методы, примеры кода и хорошую структуру API.

Производительность

Эффективная обработка запросов и минимизация задержек - ключевые аспекты. Выбор библиотеки, способной обрабатывать запросы быстро и эффективно, может повлиять на впечатление пользователей от вашего бота.

Сообщество и поддержка

Не менее важным является наличие активного сообщества и поддержки. Библиотеки с активным сообществом имеют больше шансов на быстрое устранение ошибок и поддержку новых возможностей.

python-telegram-bot

telebot - это библиотека Python для создания телеграм-ботов, разрабатываемая сообществом. Она предоставляет обширный набор функций и простой интерфейс для взаимодействия с Telegram Bot API. Эта библиотека активно поддерживается и обновляется сообществом разработчиков.

Основные особенности:

- **Поддержка асинхронности:** Библиотека позволяет вам использовать асинхронное программирование с помощью `async/await`, что делает обработку множества одновременных запросов более эффективной.
- **Обработка сообщений:** `python-telegram-bot` предоставляет удобные средства для обработки входящих сообщений, команд и событий. Вы можете легко реагировать на текстовые сообщения, изображения, видео и другие медиа-файлы.
- **Поддержка клавиатур:** Вы можете создавать интерактивные клавиатуры для взаимодействия с пользователями вашего бота.
- **Интеграция с сторонними сервисами:** Библиотека позволяет интегрировать бота с внешними сервисами, такими как базы данных и веб-сервисы.

Преимущества и недостатки:

Преимущества:

- **Активное сообщество:** `python-telegram-bot` поддерживается активным сообществом разработчиков, что гарантирует обновления и поддержку.
- **Документация:** Библиотека имеет хорошо составленную документацию, которая помогает разработчикам быстро начать работу.
- **Поддержка асинхронности:** Поддержка `async/await` позволяет эффективно управлять асинхронными операциями.
- **Обработка медиа:** `python-telegram-bot` предоставляет мощные средства для обработки изображений, аудио и других медиа-файлов.
- **Интеграция с внешними сервисами:** Библиотека упрощает интеграцию бота с другими сервисами и базами данных.

Недостатки:

- **Может показаться избыточной для простых ботов:** Для простых ботов с ограниченным функционалом, `python-telegram-bot` может показаться избыточной.

aiogram

aiogram - это высокопроизводительная и интуитивно понятная библиотека для создания телеграм-ботов на Python. Она была разработана с учетом активного участия сообщества.

aiogram предоставляет множество инструментов и функций, что делает ее идеальным выбором для создания ботов с расширенной функциональностью. Она активно поддерживается и обновляется сообществом разработчиков, и это гарантирует актуальность и надежность библиотеки.

Основные особенности:

- **Простой синтаксис:** aiogram предлагает чистый и понятный синтаксис, что упрощает начало работы и разработку ботов.
- **Поддержка асинхронности:** Библиотека разработана с учетом асинхронного программирования, что позволяет эффективно обрабатывать множество запросов одновременно.
- **Обработка сообщений:** aiogram предоставляет мощные средства для обработки сообщений, команд и событий, включая текстовые сообщения, изображения, аудио и многие другие типы данных.
- **Интерактивные клавиатуры:** Вы можете легко создавать интерактивные клавиатуры для взаимодействия с пользователями вашего бота.
- **Интеграция с сторонними сервисами:** aiogram поддерживает интеграцию с базами данных, веб-сервисами и другими внешними ресурсами, что делает ее универсальным инструментом.

Преимущества и недостатки:

Преимущества:

- **Высокая производительность:** aiogram разработана с учетом производительности, что делает ее идеальным выбором для проектов с высокой нагрузкой.
- **Асинхронное программирование:** Поддержка асинхронности позволяет эффективно управлять большим количеством одновременных запросов.
- **Богатый функционал:** Библиотека предоставляет богатый набор функций для обработки различных типов сообщений и медиа-файлов.
- **Активное сообщество:** aiogram имеет активное сообщество разработчиков, что обеспечивает быструю поддержку и обновления.
- **Интеграция:** aiogram легко интегрируется с внешними сервисами и ресурсами.

Недостатки:

- **Может потребовать более глубокого понимания асинхронного программирования:** Для некоторых разработчиков, особенно начинающих, использование асинхронности может потребовать изучения.
- **Более детальная настройка:** Иногда создание сложной функциональности может потребовать более глубокой настройки библиотеки.

1.2.2 Почему был выбран язык Python?

Основной язык для ботов — Python. Питон выбирают потому, что в его экосистеме много открытых библиотек для ботов, причем библиотек на основе искусственного интеллекта. Второй момент, почему его выбирают, — это его относительная легкость в изучении. К примеру, если вы не владеете программированием, чтобы написать собственного бота, тогда Питон будет идеальным вариантом, так как его изучить легче, чем другие языки программирования. Python универсален, поэтому боты, написанные на нем, можно интегрировать во все что угодно. Язык универсален и прост. Если перечислять преимущества ботов на Python, то стоит выделить следующие: Большая функциональность. На этом языке можно написать бот для решения различных задач. Кроссплатформенность.

1.2.3 Почему для хранения данных была выбрана база данных sqlite3?

Действительно, для хранения информации чат-бота было бы достаточно ограничиться хранением сведений непосредственно в коде программы, или отдельном текстовом файле. Однако для оптимизации, улучшения читаемости, производительности, под поставленную задачу была определена наиболее подходящая база данных.

Преимущества:

- SQLite — проще в использовании по сравнению с PostgreSQL, идеален для малых проектов.
- - легковесное решение, не требующее много времени и усилий для поддержки.
- - Продукт открыт и доступен бесплатно.

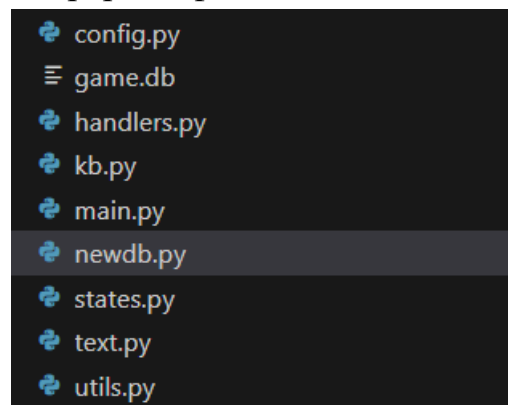
Недостатки:

- -Функционал SQLite не так велик, как у PostgreSQL, потому не всегда подходит для крупных проектов.

- - SQLite менее масштабируем, не всегда справляется с крупными проектами.
- Сообщество SQLite не так велико, как у PostgreSQL, значит, поддержка меньше.

1.3 Проектирование структуры и логики приложения

Для того чтобы обеспечить высокую читаемость кода, была смоделирована следующая структура и иерархия файлов



(Изображение 1)

Config.py – хранит стандартные настройки приложения

Game.db – файл локально развернутой базы данных

Handlers.py – файл обработки «событий» (сообщений пользователя) в телеграмм

Kb.py – файл-хранилище клавиатур для телеграмм

Main.py – файл-инициализатор приложения

Newdb.py – файл менеджер базы данных

States.py -хранилище состояний телеграмм бота

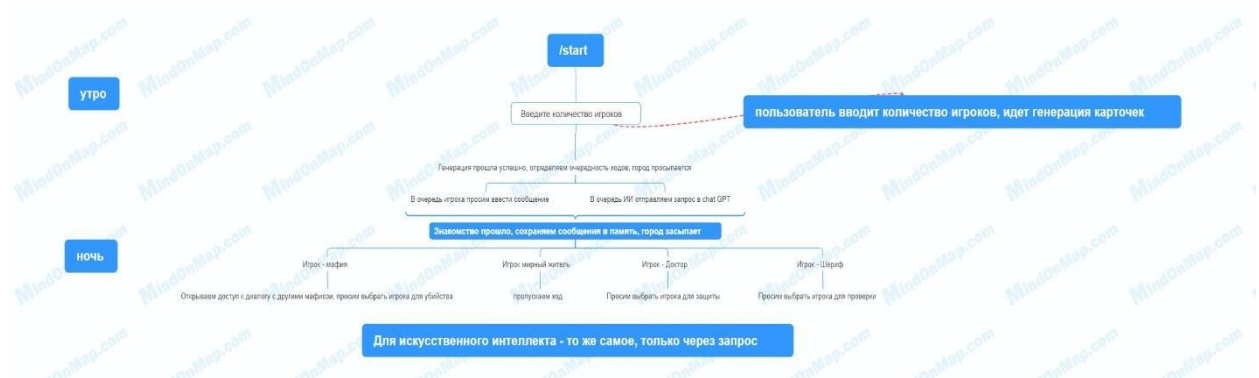
Text.py – хранилище текстовых информационных сообщений

Utils.py – файл с хранением частных методов, которые использует телеграмм бот

Глава 2 Практический блок

2.1 Моделирование хода диалога бота

Для того чтобы определить количество и фильтры хэндлеров(обработчиков сообщений пользователя) для начала необходимо построить структурную схему бота. С использованием бесплатных сервисов, автор определил логику программы, количество необходимых обработчиков.



Изображение (2)

Проанализировав Изображение 2. можно смоделировать следующий ход диалога:

- Пользователь запускает игру по нажатии кнопки «/start»
- Ведущий запрашивает количество игроков
- Вне обработчиков выполняются функции по созданию сводной таблицы сессии игрока с ключом в виде ID пользователя telegram
- Приложение переходит в состояние «день», происходит знакомство участников в очередь игрока, пользователь прописывает сообщение, доступное всем ботам сессии для последующего анализа, вне его очереди в сервис искусственного интеллекта
- Приложение переходит в состояние «ночь», происходит реализация инструментов ролей персонажей, в зависимости от очереди, либо игрок, основываясь на полученных данных выбирает объект инструмента, либо сервис искусственного интеллекта, для роли «Мафия», в случае если игрок также относится к этой роли, будет доступен многоуровневый чат с ходом выбора объекта
- Завершение сессии

2.2 Разработка обработчиков событий, функций событий

Основываясь на модели хода диалога, определим обработчики и внешние функции для обработчиков:

```
def extract_name_from_response(response: str, player_names: list) -> str:
    # Регулярное выражение для извлечения имени из сообщения
    match = re.match(r"Я выбрал - ([\w\sа-яА-ЯёЁ]+)", response)
    if match:
        chosen_name = match.group(1).strip()
        if chosen_name in player_names:
            return chosen_name
    return None

async def
gpt_us_send_mess(ComparedMessage, provider: g4f.Provider.BaseProvider):
    try:
        response = await g4f.ChatCompletion.create_async(
            model=g4f.models.default,
            messages=[{"role": "user", "content": ComparedMessage}],
            provider=provider,
        )
        chat_gpt_response = response
        return chat_gpt_response
    except Exception as e:
        print(f"{provider.__name__}:", e)

async def gpt_us(ComparedMessage):
    tasks = [
        gpt_us_send_mess(ComparedMessage, provider) for provider in _providers
    ]
    for task in asyncio.as_completed(tasks):
        response = await task
        if response is not None:
            cleaned_response = re.sub(r"\$@\$.+?\$@\$", "", response)
            return cleaned_response
    return None

async def generate_players(player_count, game_key):
```

```

Gamer = random.randint(1, player_count)
Gamer = 1
# Определение ролей
mafia_count = int(player_count // 3.5)
roles = ['Мирный житель'] * (player_count - mafia_count)
roles.extend(['Мафия'] * mafia_count)
roles.extend(['Доктор', 'Шериф'])
names = ["Фетучинни", "Макиавелли", "Шегги", "Базилик",
        "Фелиция", "Саманта", "Роуз", "Герда", "Курт",
        "Савелий", "Шершень", "Элизабет", "Ханна",
        "Виолетта", "Чарльз", "Грегори", "Витто",
        "Альварэ", "Донателло", "Альма", "Сальваторе"]
for i in range(player_count):
    random.shuffle(roles)
    random.shuffle(names)

# Генерация данных игроков
players = []
for i in range(player_count):
    player = {
        'game_key': game_key,
        'queue': i + 1,
        'role': roles[i],
        'suit': 'Черный' if roles[i] == 'Мафия' else 'Красный',
        'con1': [],
        'con2': [],
        'con3': [],
        'is_gamer': True if i == Gamer else False,
        'name': names[i]
    }
    players.append(player)
return players

async def add_generated_players_to_db(db_manager, player_count):
    players = await generate_players(player_count)
    for player in players:
        db_manager.add_player(player['queue'], player['role'], player['suit'],
                               player['con1'], player['con2'], player['con3'],
                               player['isgamer'], player['name'])

```


функция	Описание метода
gpt_us	Метод для отправки запроса на сервера искусственного интеллекта. Входные данные: ● ComparedMessage – промт ● Provider – сервер Выходные данные: ● Cleaned_response – обработанный регулярными выражениями ответ сервера. ● Особенности: ● при отправке запроса, может возникать «загрязнение ответа» индивидуальным идентификатором чата, при помощи библиотеки регулярных сообщений, метод анализирует ответ и исключает идентификатор чата из итогового ответа
generate_players	Метод генерации карточек игрока. Входные данные: ● player_count – количество карточек для генерации ● game_key – уникальный идентификатор сессии Выходные данные: ● players – список с карточками игрока

extract_name_from_response	Метод определения выбора искусственного интеллекта для состояния «ночь» Входные данные: ● response – ответ сервера ● player_names – список имен участников одной сессии Выходные данные: ● chosen_name – выделенное имя из ответа сервера Особенности: ● поскольку сервер – искусственный интеллект, - нельзя точно быть уверенным, что оно определит имя участника, для корректной работы состояния, было определено регулярное выражение для «отлова» имени, в промт внедрен запрос указать в конце сообщения ответ в виде, которое поддерживает регулярное выражение
----------------------------	--

(Таблица 2.)

С полным кодом обработчиков можно ознакомиться в коде файла handlers.py, в рамках реализации приложения были произведены пробные запуски ресурса, в ходе которых было определено, что приложение работает корректно, обработчики полностью выполняют потребный функционал.

Метрика возможных ошибок:

Ошибка	Решение
Ответ сервера на иностранном языке, «загрязнение» ответа	При использовании определенного ресурса искусственного интеллекта, ответ может выводиться на

	иностранном языке или же содержать в себе слова не родственные запросу. Для «очистки» ответа был модернизирован промт, а также применены регулярные выражения для отлова возможных «загрязнителей»
Ввод текстовых сообщений пользователем с типом данных, который отличается от требуемого для реализации функции	Если пользователь вводит цифровое значение вместо буквенного или же наоборот, в ходе работы приложения могут возникать ошибки. Для того, чтобы устранить некорректный ввод пользователем, при запросе сообщений всплывает информационное окно с указанием необходимого формата ответа, запрос был зациклен при помощи машины состояний FSM Решение при запросе дневного сообщения для анализа искусственным интеллектом не представляется возможным и, в рамках игры, расценивается на уровне с мета-геймингом, боты искусственного интеллекта могут указывать на «странное» поведение игрока
Ошибка генерации карточек игроков	При работе с базой данных, включением сформированных карточек, может возникнуть техническая ошибка, связанная с нестабильным соединением интернета или же некорректным определенным состоянием игры.

	При возникновении ошибки сервис предлагает перезагрузить сессию, формирование сессии с ключом в виде ID происходит с проверкой на наличие сведений о ранней сессии, при обнаружении такая сессия удаляется
Ошибка определения выбранного участника искусственным интеллектом	Поскольку определение выбранного искусственным интеллектом производится за счет регулярного выражения, иногда формат ответа может с ним не совпадать, в таком случае запрос перенаправляется

(Таблица 3.)

Заключение

На этапе тестирования приложения автор выявил особенности взаимодействия искусственного интеллекта со своими дочерними участниками сессии и пользователем, работа приложения показывает высокий уровень стратегических навыков искусственного интеллекта, одновременно с этим, сложную модель поведения, которая требует корректных и точных запросов в сервис. Автор видит большой потенциал во внедрении элементов искусственного интеллекта в самые разные сферы. На текущий момент используя ресурсы, который предоставляет ресурс ChatGPT к реализации доступно множество успешных коммерческих продуктов, сокращение потребного для нахождения решений времени, автор сравнивает создание искусственного интеллекта с изобретением книги, это новый этап в жизни человека и информации. Если внедрить его в жизнь студента и учебный процесс, это позволит повысить скорость потребления информации и её усвоения. Даже на примере игры-стратегии можно выявить этот большой потенциал, к моменту написания заключения в приложение заложен большой потенциал для выхода в публичный доступ для ознакомления молодых разработчиков с методами применения в своих собственных программах. Используя модели, которые были разработаны, исключив ошибки, которые были выявлены в метрике, читатель может смоделировать собственный интерфейс для близкого знакомства с ресурсом.

Задачи курсовой работы были полностью выполнены, их решение в полной мере отражено в теоретической и практической части:

- Изучена специальная литература по представленной теме
- Рассмотрены ключевые понятия и определения
- Представлена компьютерная модель и финальный код программы

СПИСОК ЛИТЕРУРЫ

1. Тощакон А.В. // Aiogram 3.7.0 documentation / Тощакон А.В. - URL: <https://docs.aiogram.dev/> (дата обращения: 23.05.2024).
2. Арина Семчугова. // Промпты для ChatGPT — инструкции / Арина Семчугова. - URL: <https://www.setters.media/post/chatgpt-na-maksimalkah-instrukciya-i-60-promtov-na-russkom-yazyke> (дата обращения: 23.05.2024).
3. Friedl, Jeffrey. // re — Regular expression operations / Friedl, Jeffrey. - URL: <https://docs.python.org/3/library/re.html> (дата обращения: 23.05.2024).
4. Антон Яценко. // Чат-боты в Telegram на Python и Aiogram / Антон Яценко. - URL: <https://skillbox.ru/media/code/chatboty-v-telegram-na-python-chast-1/> (дата обращения: 23.05.2024).
5. UPROGER | Программирование. // Простое руководство по ChatGPT API с использованием Python / Анонимный автор. - URL: <https://uproger.com/prostoe-rukovodstvo-po-chatgpt-api-s-python/?ref=vc.ru> (дата обращения: 23.05.2024).
6. Кабанова Н.В., Кленина Н.В., Чекмарёв В.В. Моделирование компьютерных игр: Учебное пособие. — М.: МФТИ, 2018. — 224 с.
7. Дейкстра Э.С. Избранные лекции по программированию. — М.: Мир, 2004. — 480 с.
8. Рассел С., Норвиг П. Искусственный интеллект: современный подход. — СПб.: Питер, 2006. — 1168 с.
9. Андерс Т. Искусственный интеллект в играх: Методы математического моделирования, обучения и поиска. — М.: ДМК Пресс, 2019. — 352 с.
10. Гемблер Р., Матушкин К. Принципы и методы искусственного интеллекта: Учеб. пособие. — М.: Изд-во МГУ, 2017. — 296 с.
11. Грустель А.А., Ионов С.А. Элементы логического программирования: Учеб. пособие. — М.: Наука, 2005. — 240 с.
- 11.. Геннадий Коротченко, Александр Цицулин. Создание компьютерных игр: практ. рук. для начинающих. — СПб.: БХВ-Петербург, 2017. — 304 с.
12. Ивонин Д.В., Хапов Д.А. Программирование искусственного интеллекта на языке Ада. — М.: ДМК Пресс, 2015. — 224 с.